

OBLIGATORIO 1 : ESTRUCTURA DE DATOS Y ALGORITMOS 2 2026

INTEGRANTES:

- Bruno Rosa - Nro estudiante(318321)
- Shaiel Tito Virgilio - Nro estudiante(305773)

DOCENTES:

- Horacio Abalos
- Joaquin vigna

EXPLICACIONES DE LOS EJERCICIOS:

EJERCICIO 1:

Para este ejercicio utilizamos una tabla de hash para modelar las llaves disponibles. Lo que hicimos fue recorrer la cadena de entrada una única vez en orden, donde las posiciones pares representan llaves y las impares puertas. En cada paso, se mantiene en la tabla hash la cantidad de llaves disponibles de cada tipo. Cuando se encuentra una llave, se inserta o incrementa su contador, y cuando se encuentra una puerta, se verifica si existe una llave correspondiente, si existe se utiliza (decrementando su cantidad), y si no existe se contabiliza como una llave adicional necesaria.

La tabla fue implementada utilizando hash cerrado con direccionamiento lineal para resolver colisiones. Esto nos permite realizar las operaciones de búsqueda, inserción y actualización en tiempo promedio $O(1)$.

Respecto a la complejidad, la cadena se recorre exactamente una vez y en cada iteración únicamente se realizan operaciones constantes sobre la tabla hash. Por lo tanto, el costo total es $O(n)$, cumpliendo con la complejidad pedida en la letra.

EJERCICIO 2:

Para resolver este ejercicio utilizamos dos tablas hash cerradas con doble hashing, la primera tabla se utilizó para acumular los puntajes totales de cada participante y calcular el puntaje máximo alcanzado. La segunda tabla la creamos para simular el proceso en orden y detectar el primer participante que alcanza o supera ese máximo acumulado durante la segunda pasada.

En la primera corrida, insertamos a cada participante en la primera tabla acumulando sus puntajes y verificamos si supera el máximo actual para mantener actualizado el valor de maxScore.

Una vez conocido el puntaje máximo final, realizamos una segunda pasada recorriendo nuevamente los participantes en el orden original de aparición. En esta etapa solamente procesamos aquellos participantes cuyo puntaje total final coincide con maxScore.

Para cada uno de ellos, acumulamos progresivamente sus puntajes en la segunda tabla hash. Luego de cada actualización verificamos si su puntaje acumulado en esta segunda simulación alcanza o supera el máximo calculado anteriormente. El primer participante que cumple esta condición es el que debe imprimirse, finalizando la ejecución.

La utilización de dos tablas hash nos permitió mantener operaciones promedio $O(1)$ para búsquedas, inserciones y actualizaciones, evitando recorridos adicionales sobre los datos.

Al realizar pruebas con casos donde todos los puntajes son negativos, nos dimos cuenta que la inicialización del máximo en 0 producía resultados incorrectos. Esto se corrigió utilizando el menor valor posible de un entero (INT_MIN) como valor inicial.

Respecto a la complejidad, realizamos dos recorridos lineales sobre la entrada y en cada operación únicamente ejecutamos accesos constantes sobre las tablas de hash. Como no se realizan ciclos anidados sobre los participantes ni búsquedas lineales adicionales, el costo total termina siendo proporcional únicamente a la cantidad total de registros procesados. Por lo tanto, la complejidad final del algoritmo es $O(n)$.

EJERCICIO 3:

Para resolver este ejercicio implementamos una Cola de Prioridad utilizando un Max-Heap. la representación del Heap en el arreglo, optamos por comenzar desde la posición 1, dejando la posición 0 sin utilizar (como vimos en clase). Esta decisión de diseño simplifica las cuentas. Para cualquier nodo ubicado en la posición i , su hijo izquierdo se encuentra en la posición $i * 2$, su hijo derecho en $i * 2 + 1$, y su padre en la posición $i / 2$.

Primero, recorreremos la entrada de datos insertando a cada paciente en la cola de prioridad. En cada inserción, el nodo se coloca al final del arreglo y se aplica una operación de "flotado", comparando al paciente con su nodo padre y subiéndolo en el árbol si su prioridad es mayor, en caso de empate se mira la hora de llegada, y se pone como padre el que llegó antes esta antes.

Una vez cargados todos los pacientes, vamos sacando desde la raíz (Tope) del Heap, que siempre contiene al paciente de mayor prioridad. Luego de cada extracción, movemos el último elemento a la raíz y aplicamos hundido para restaurar el orden correcto.

Complejidad, tanto insertar como extraer un elemento cuesta $O(\log n)$. Cómo realizamos estas operaciones para los n pacientes, la complejidad total del algoritmo es $O(n \log n)$

EJERCICIO 4:

Para este ejercicio utilizamos un grafo dirigido y ponderado implementado mediante listas de adyacencia para representar las conexiones entre ciudades junto con sus costos. Como los nombres de las ciudades eran strings y el grafo trabaja internamente con índices numéricos, utilizamos una tabla hash cerrada para mapear cada ciudad a un identificador entero único.

Durante la lectura de las conexiones, primero verificamos si cada ciudad ya existía en la tabla hash utilizando `buscarPuntaje`. Si no existía, la insertamos asignándole un nuevo índice incremental mediante `proxIndice`. De esta manera logramos transformar los nombres de ciudades en vértices numéricos sin perder eficiencia en las búsquedas.

Luego de obtener los índices correspondientes de cada ciudad, insertamos las aristas en el grafo utilizando `insertarArista(u, v, peso)`, donde el peso representa el costo asociado a la conexión.

Para resolver el problema utilizamos el algoritmo de Dijkstra, ya que necesitábamos calcular el camino mínimo entre un origen y un destino en un grafo con pesos positivos.

Mantuvimos las tres estructuras auxiliares al igual que vimos en clase:

- costos, para guardar la menor distancia conocida hacia cada vértice.
- visitados, para evitar reprocesar vértices ya confirmados.
- vengoDe, para almacenar el vértice anterior del camino mínimo encontrado.

Inicializamos todas las distancias en infinito mediante `RellenarVec`, excepto el vértice origen, cuyo costo se establece en 0.

Para seleccionar eficientemente el próximo vértice con menor costo utilizamos un `MinHeap`. Insertamos inicialmente el origen en el heap y luego repetimos el siguiente proceso mientras el heap no estuviera vacío:

- Extraer el vértice con menor distancia conocida.
- Si todavía no había sido visitado, marcarlo.
- Recorrer todos sus adyacentes.
- Relajar las aristas verificando si pasar por el vértice actual mejora el costo conocido del vecino.

La relajación se realiza con la condición $\text{costos}[\text{verticeAdy}] > \text{costos}[\text{verticeActual}] + \text{pesoAdy}$

Si encontramos un camino mejor, actualizamos el costo, guardamos el vértice anterior en `vengoDe`, e insertamos nuevamente el vecino en el heap con su nuevo costo.

Al finalizar, si el costo del destino continúa en INT_MAX, significa que no existe camino entre las ciudades y se imprime -1. En caso contrario, se imprime el costo mínimo encontrado.

Respecto a la complejidad, las búsquedas e inserciones en la tabla hash tienen costo promedio $O(1)$, por lo que mapear ciudades no afecta significativamente el orden total.

En el algoritmo de Dijkstra, cada vértice puede insertarse en el heap, cada extracción e inserción cuesta $O(\log V)$, y cada arista se recorre una vez durante las relajaciones. Por lo tanto, la complejidad total resulta: $O((V+A)\log V)$

cumpliendo con el orden esperado para Dijkstra utilizando min heap y listas de adyacencia.

EJERCICIO 5:

Para este ejercicio utilizamos un grafo no dirigido implementado mediante listas de adyacencia para representar las incompatibilidades entre estudiantes. Cada vértice representa un estudiante y cada arista indica que dos estudiantes no pueden pertenecer al mismo equipo.

La resolución la realizamos utilizando DFS recursivo junto con un arreglo de colores para simular la división en dos equipos y si ya estaban visitados. En el arreglo colores utilizamos:

-1 para indicar que un vértice todavía no fue visitado.

0 para representar el equipo A.

1 para representar el equipo B.

Primero construimos el grafo leyendo todas las incompatibilidades e insertando cada arista. Luego reservamos memoria para el arreglo de colores e inicializamos todas sus posiciones en -1, ya que inicialmente ningún estudiante pertenece a un equipo.

La lógica principal se encuentra en la función DFS. Cuando visitamos un vértice, le asignamos el color recibido por parámetro y luego recorremos todos sus adyacentes mediante la lista de adyacencia obtenida con `obtenerAdyacentes(vertice)`.

Durante el recorrido ocurren dos casos:

Si el vértice vecino todavía no fue visitado, llamamos recursivamente a DFS con el color opuesto mediante `1 - colorAct`, esto hace que vaya cambiando entre 1 y 0 cada iteración.

Si el vecino ya fue visitado y tiene el mismo color que el vértice actual, significa que dos estudiantes incompatibles quedarían en el mismo equipo, por lo que retornamos falso.

La función DFS retorna true únicamente si toda la componente conexa puede colorearse correctamente sin conflictos.

En el main recorremos todos los vértices del grafo porque el mismo puede no ser conexo. Cada vez que encontramos un vértice sin visitar, iniciamos un nuevo DFS desde ese vértice. Si alguna ejecución retorna falso, marcamos que el grafo no es bipartito y finalizamos el proceso.

La complejidad temporal es $O(V+A)$ porque cada vértice se visita una única vez, y cada arista se recorre una sola vez durante el DFS.

La complejidad espacial también es $O(V+A)$ debido al almacenamiento del grafo mediante listas de adyacencia y al arreglo auxiliar de colores.

Tabla de resultados:

Problema	Resultado
1	Completo
2	Completo
3	Completo
4	Completo
5	Completo